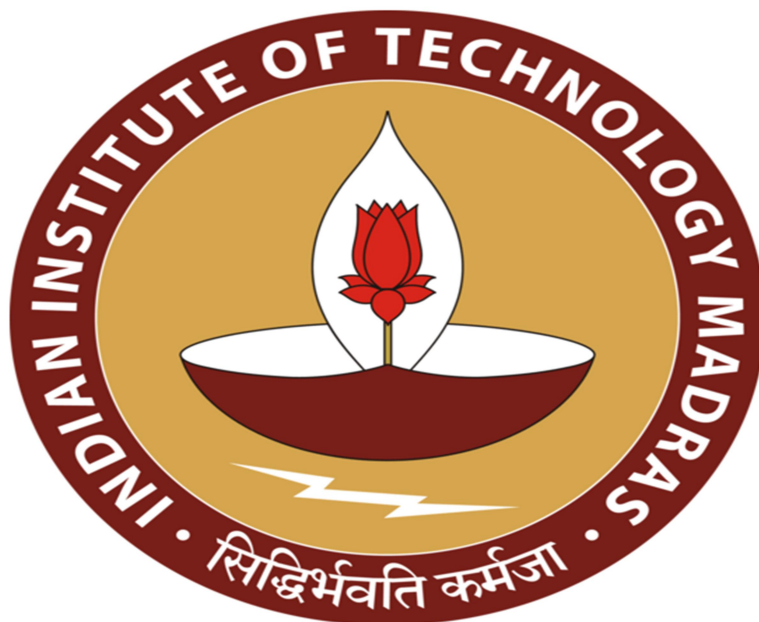


SOFTWARE ENGINEERING PROJECT MAY-2026
MILESTONE 2 DELIVERABLES FOR COMMUNITY SERVICES PLATFORM
SUBDOMAIN: APARTMENT ASSOCIATION

Submitted By

Team Name: Sputnik

Team Code: MAY2026-Team-095



IITM Online BS Degree Program,
Indian Institute of Technology, Madras, Chennai

Tamil Nadu, India, 600036

SMART SOCIETY COMPLAINT RESOLUTION SYSTEM

Project Details

Chosen Problem Statement: Community Services Platform

Subdomain: Apartment Association

Software Name: Smart Society Complaint Resolution System

Date of Submission: 22-07-2026

Team Details

Team Name: Sputnik

Team Code: MAY2026-Team-095

Name	Roll Number	Role
P Baskaran Nadar	24f2008200	Project Manager
Rahul Khanna GK	24f1002269	Backend Developer
Jishan Kapoor	24f1002364	Frontend Developer
Annapoorani Muthuveerappan	23f1001421	Scrum Master
Roshan Ashwin Singh	22f1000684	Quality Assurance

DESIGN OF COMPONENTS

System Components

The System has been designed into two useful categories: **user-facing views** such as Resident, Society Manager, Maintenance Staff, Chairman, Secretary, Management Committee Member, Auditor, and Security Personnel; and **shared functional/system components** such as Authentication and Authorization, Complaint Management, Staff Assignment, Billing and Payment, Notifications, Announcements, Feedback, and Dashboard/Reporting

Design of Components

- **Resident View (APIs/User Activities)**
 - Register and log in to the system
 - View and update profile details
 - Create a new maintenance complaint
 - Edit an existing complaint where permitted
 - View submitted complaints and their current status
 - Track the progress of maintenance requests
 - View assigned maintenance staff details
 - Reopen a closed complaint where permitted
 - Rate the service and provide feedback after resolution
 - View society announcements and notices
 - Receive notifications regarding complaint and service updates
 - Display notification updates dynamically in the resident interface
 - Use Redis to support fast retrieval of frequently accessed notification data where required
 - Use Celery tasks for asynchronous delivery of complaint and service notifications
- **Society Manager View (APIs/Management Activities)**
 - Log in to the manager dashboard
 - View and manage resident details

- View all maintenance complaints
- Filter complaints by category, priority, status, or date
- Assign complaints to appropriate maintenance staff
- Reassign complaints where required
- Monitor pending, in-progress, and completed maintenance work
- Verify complaint resolution
- Manage maintenance staff information
- Create and publish society announcements
- View maintenance and operational summaries
- Review resident feedback and service ratings
- **Maintenance Staff View (APIs/Work Activities)**
 - Log in to the maintenance staff dashboard
 - View assigned maintenance jobs
 - View complaint details associated with assigned work
 - Accept or acknowledge assigned work
 - Update work status
 - Add work remarks and resolution notes
 - Mark maintenance work as completed
 - View previously completed jobs
 - Report inability to complete an assigned task where applicable
 - Receive notifications regarding new work assignments
 - Use Celery tasks for asynchronous delivery of new work assignment notifications
- **Secretary View (APIs/Administrative Activities)**
 - Log in to the Secretary dashboard
 - View resident and society records

- View and monitor maintenance complaints
- Create and publish society announcements and notices
- Schedule society meetings and events
- Monitor maintenance work progress
- View complaint resolution details
- Receive and post notifications regarding important society activities
- Use Celery tasks for asynchronous delivery of important society activity notifications
- **Authentication and Authorization (APIs/System Activities)**
 - Register authorized users
 - Authenticate users during login
 - Validate user credentials
 - Identify the role of the authenticated user
 - Redirect users to their respective dashboards
 - Use HTML, CSS, and JavaScript to implement role-specific frontend pages, navigation, and redirection
 - Restrict unauthorized access to protected pages
 - Maintain user sessions or authentication tokens
 - Use Redis for temporary session, token, or frequently accessed authentication data where required
 - Support role-based access control
 - Handle invalid credentials and unauthorized requests
- **Complaint Management (APIs/System Activities)**
 - Create a new maintenance complaint
 - Generate a unique complaint identifier
 - Store complaint category and description
 - View complaint details

- Retrieve complaint history
- Update complaint information where permitted
- Track complaint status
- Maintain complaint status history
- Close a resolved complaint
- Reopen a closed complaint where permitted
- Retrieve complaint records from the database
- Use Redis caching for frequently accessed complaint data where appropriate to improve retrieval performance
- **Staff Assignment and Work Management (APIs/System Activities)**
 - Add new maintenance staff
 - Remove a maintenance staff based on poor ratings
 - Retrieve available maintenance staff
 - Use Redis caching for frequently accessed staff availability data where appropriate
 - Assign complaints to appropriate maintenance staff
 - Match staff expertise with complaint categories
 - Reassign maintenance work where necessary
 - Store work assignment details
 - Track assigned maintenance work
 - Update work progress
 - Record completion details and resolution notes
 - Maintain work assignment history
- **Notification Management (APIs/Background Activities)**
 - Generate notifications for important system events
 - Use Celery tasks to process notification jobs asynchronously in the background

- Notify residents when a complaint is created
- Queue complaint confirmation notifications as Celery background tasks
- Notify maintenance staff when new work is assigned
- Queue work assignment notifications as Celery background tasks
- Notify residents when complaint status changes
- Queue complaint status update notifications as Celery background tasks
- Notify managers about important pending activities
- Use scheduled Celery tasks for reminders about pending activities where required
- Store notification records
- Use Redis for temporary notification caching and fast retrieval of unread notification data where appropriate
- Retrieve unread notifications
- Mark notifications as read
- Deliver alerts to relevant users
- Use Celery workers for background notification processing and Redis as the message broker for queued tasks
- **Announcement and Notice Management (APIs/System Activities)**
 - Create society announcements
 - Publish society notices
 - View active announcements
 - Edit announcements where permitted
 - Archive outdated announcements
 - Display announcements to relevant users
 - Store announcement publication dates
 - Maintain announcement history
- **Feedback and Rating Management (APIs/System Activities)**

- Allow residents to submit feedback after service completion
- Allow residents to rate completed maintenance work
- Store feedback comments
- Associate feedback with the relevant complaint
- Allow authorized users to review feedback
- Maintain service rating history
- Retrieve feedback records from the database
- Use HTML, CSS, and JavaScript to display feedback and rating information in the relevant dashboards
- **Dashboard and Reporting (APIs/Export Activities)**
 - Display role-specific dashboard information
 - Develop role-specific dashboard interfaces using HTML, CSS, and JavaScript
 - Show complaint counts by status
 - Show pending and completed maintenance work
 - Display resident-specific complaint summaries
 - Display staff-specific work assignment summaries
 - Display management-level operational summaries
 - Generate administrative reports
 - Generate audit reports
 - Export relevant reports where permitted
 - Retrieve summarized information from the database
 - Use Redis caching for frequently accessed dashboard summaries and report data where appropriate

CLASS DIAGRAM

The class diagram illustrates the object-oriented architecture of the **Smart Society Complaint Resolution System**. It represents the core classes, their attributes, operations, and relationships involved in the complaint management workflow. The design models the interactions between different user roles, namely **Residents**, **Maintenance_Staff**, and **Administrators (Manager, Secretary, Chairman)**, which inherit common properties and functionalities from the **User** class.

The diagram also depicts the relationships between the primary system entities, including **Complaint**, **Category**, **Complaint Update**, **Attachment**, **Feedback**, and **Notification**. These classes collectively support complaint registration, categorization, assignment, progress tracking, resolution, feedback collection, attachment management, and user notifications. This class diagram serves as the structural blueprint for implementing the backend business logic, database schema, and object interactions of the **Smart Society Complaint Resolution System**.

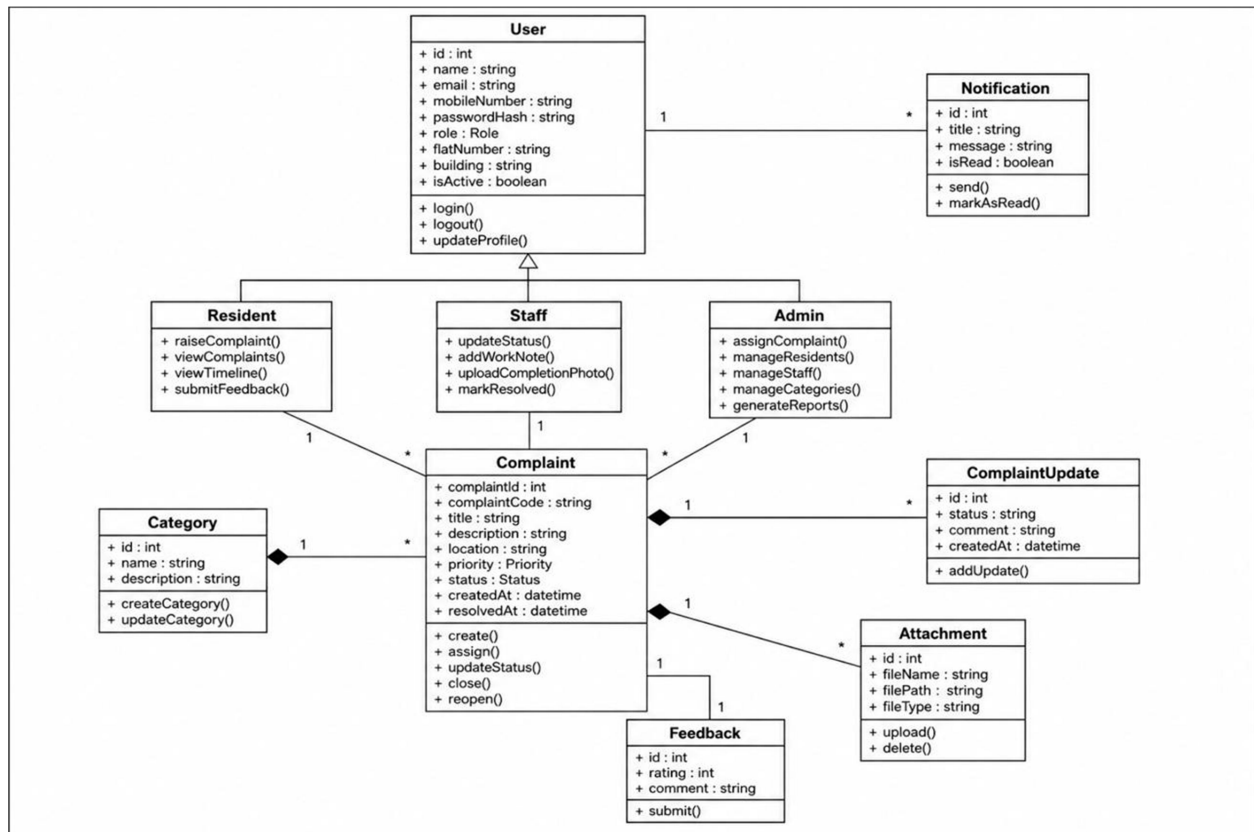


Diagram 1: Class Diagram for Smart Society Complaint Resolution System

DATABASE DESIGN – ER DIAGRAM

The database design diagram represents the relational database schema of the Smart Society Complaint Resolution System. It illustrates the database tables, their attributes, primary keys, foreign keys, and relationships that support the complaint management workflow. The schema includes entities such as Users, Complaints, Categories, Subcategories, Complaint Updates, Attachments, Feedback, Notifications, Notification Preferences, and Audit Logs, along with reference tables for user roles, complaint priorities, statuses, and notification types. The design ensures data integrity, efficient data retrieval, and proper relationships between system entities, providing a robust foundation for implementing the backend using Flask, SQLite, Redis, and Celery.

SMART SOCIETY – Complaint Resolution System
 Database Design Diagram

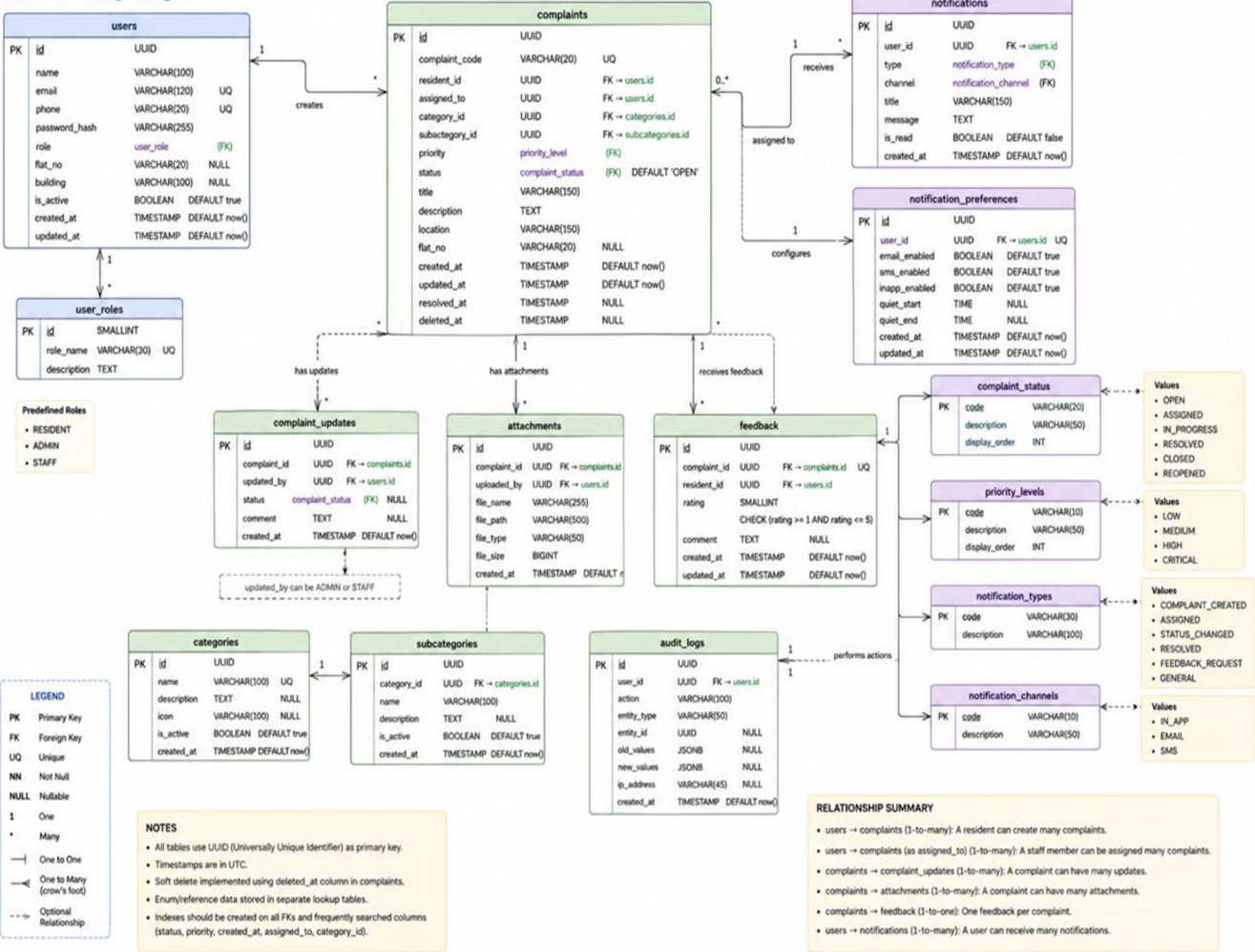


Diagram 2: Database Diagram for Smart Society Complaint Resolution System

PROJECT SCHEDULING TOOLS

TOOL	PURPOSE
Jira	Sprint Planning, Task management, Kanban Board, Backlog Management, Workflow Tracking, Issue tracking and Team collaboration.
Excel and Gantt Chart	Project scheduling, Milestone planning, task sequencing, dependency visualization, and Timeline management.
Google Meet	SCRUM meetings, Sprint planning, Project discussions, and Stakeholder communication.
GitHub	Version control, collaborative development, branch management, and repository maintenance.

SPRINT SCHEDULE

Sprint 1 – Milestone 1: Requirements Engineering

Duration: Before 28 June 2026

- Identify the primary, secondary, and tertiary users of the application.
- Conduct stakeholder interviews and observations to identify existing pain points without directly suggesting software features.
- Collect and maintain supporting evidence through audio and video recordings where applicable.
- Prepare user stories as per the SMART format.
- Review, refine, and validate the documented user stories.
- Prepare and submit the Milestone 1 report, including stakeholder analysis, user research, identified pain points, and SMART user stories.

Sprint 2 – Milestone 2: Project Planning and System Design

Duration: 28 June 2026 – 22 July 2026

- Finalize the project title, branding, technology stack, and development tools.
- Prepare the overall project schedule, sprint schedule, and Gantt chart.
- Configure the Jira project board, workflow, and task assignments.
- Design the system components and software architecture.
- Prepare the UML Class Diagram and Database Design (ER) Diagram.
- Develop the major frontend pages using HTML, CSS, JavaScript, and Vue.js (CDN).
- Connect the frontend pages through routing and navigation.
- Document the SCRUM meeting schedule and meeting minutes.
- Compile and submit the Milestone 2 report, including all required diagrams, schedules, screenshots, and frontend deliverables.

Sprint 3 – Milestone 3: API Development (Sprint 1)

Duration: 20 July 2026 – 02 August 2026

- Identify and prioritize the core REST API endpoints required by the system.
- Design Swagger-compatible YAML documentation for all developed APIs.

- Implement the APIs using Flask with proper validation, error handling, and standardized responses.
- Map each API endpoint to the corresponding user stories.
- Develop unit tests using pytest for all implemented APIs.
- Perform API testing and document the test cases and results.
- Track development progress and reported issues using GitHub.

Sprint 4 – Milestone 4: System Integration and Testing (Sprint 2)

Duration: 02 August 2026 – 12 August 2026

- Integrate the frontend with the backend APIs.
- Implement the core user workflows across all system modules.
- Design and execute comprehensive functional test cases.
- Record the inputs, expected outputs, actual outputs, and test results for each API.
- Resolve integration issues and functional defects identified during testing.
- Collect user feedback and refine the implemented features accordingly.
- Update the project documentation and README file.

Sprint 5 – Milestone 5: Final Testing and Documentation

Duration: 12 August 2026 – 22 August 2026

- Demonstrate the complete working application.
- Execute the final pytest test suite and document the results.
- Resolve any remaining bugs and improve overall system stability.
- Collect, review, and summarize user feedback.
- Prepare the final project report, incorporating all milestone deliverables and supporting screenshots.
- Prepare the project presentation and demonstration materials.

Sprint 6 – Final Submission

Submission Date: 23 August 2026

- Perform a final review of the complete project, including source code and documentation.
- Finalize the demonstration video and upload it to the required platform.
- Proofread the final report and verify all implementation and execution instructions.

- Prepare the final source code package along with the README file.
- Submit all required deliverables, including the project report, source code, presentation, demonstration video, and supporting documents.

SCRUM MEETING SCHEDULE AND MINUTES

SCRUM meetings were conducted on the scheduled dates through Google Meet during evening sessions (21:00 to 22:00 Hrs) to review project progress, discuss challenges, assign responsibilities, and plan subsequent milestones.

Minutes of SCRUM Meeting - June 20, 2026:

- Team members introduced themselves and discussed their respective roles, responsibilities, and areas of expertise.
- Explored multiple project domains and finalized the **Apartment Association / Complaint Resolution System** as the project topic.
- Brainstormed the overall complaint resolution workflow and system functionality.
- Identified key challenges faced by apartment residents, including delayed complaint resolution, lack of transparency, ineffective communication, and limited accountability.

Minutes of SCRUM Meeting - June 24, 2026:

- Reviewed the overall project progress and clarified individual responsibilities among team members.
- Discussed project compliance requirements, including obtaining consent for audio and video recordings during stakeholder interactions.
- Conducted a detailed discussion on the proposed system architecture and overall design approach.
- Reviewed the SMART guidelines for writing effective user stories.
- Identified and finalized the project's primary, secondary, and tertiary user groups.

Minutes of SCRUM Meeting - June 26, 2026:

- Analysed the information collected during stakeholder interviews and documented the key pain points.
- Drafted **17 SMART user stories** covering the complete complaint management workflow.
- Reviewed the user stories and project documentation for completeness and consistency.

- Initiated **Milestone 2**, focusing on frontend wireframing, page layouts, and the overall user interface structure.

Minutes of SCRUM Meeting - July 1, 2026:

- Finalized the project's technology stack, including **Flask, HTML, CSS, JavaScript (Vue via CDN), SQLite, Redis, and Celery**.
- Reviewed and validated the proposed user interface wireframes.
- Discussed the design and layout of the application's frontend pages.
- Planned the overall project timeline, sprint schedule, and development workflow.

Minutes of SCRUM Meeting - July 3, 2026:

- Presented and reviewed the frontend prototype developed by the team.
- Discussed the **Jira** project structure, task allocation, Kanban workflow, and progress tracking.
- Emphasized the importance of maintaining a well-organized **GitHub** repository with proper version control practices.
- Agreed to develop the frontend and backend modules in parallel to improve development efficiency.

Minutes of SCRUM Meeting - July 8, 2026:

- Discussed the APIs required to facilitate communication between the frontend and backend.
- Clarified the differences between the UML Class Diagram and the Database Design (ER) Diagram.
- Reviewed the current progress of frontend development and identified pending tasks.
- Discussed the frontend project structure, including the organization of pages, reusable components, assets, and routing.

Minutes of SCRUM Meeting - July 9, 2026:

- Discussed the APIs planned for implementation during the upcoming development sprints.
- Reviewed the initial drafts of the **Gantt Chart, SCRUM Meeting Minutes, Class Diagram, and Database Design Diagram**.
- Proposed and discussed additional API endpoints required to support the defined user stories.
- Clarified team members' queries regarding the upcoming milestones and project deliverables.

- Agreed to consolidate and finalize all documentation required for **Milestone 2** before submission.

Minutes of SCRUM Meeting – July 10, 2026:

SCREENSHOT OF GANTT CHART ALONG WITH DESCRIPTION

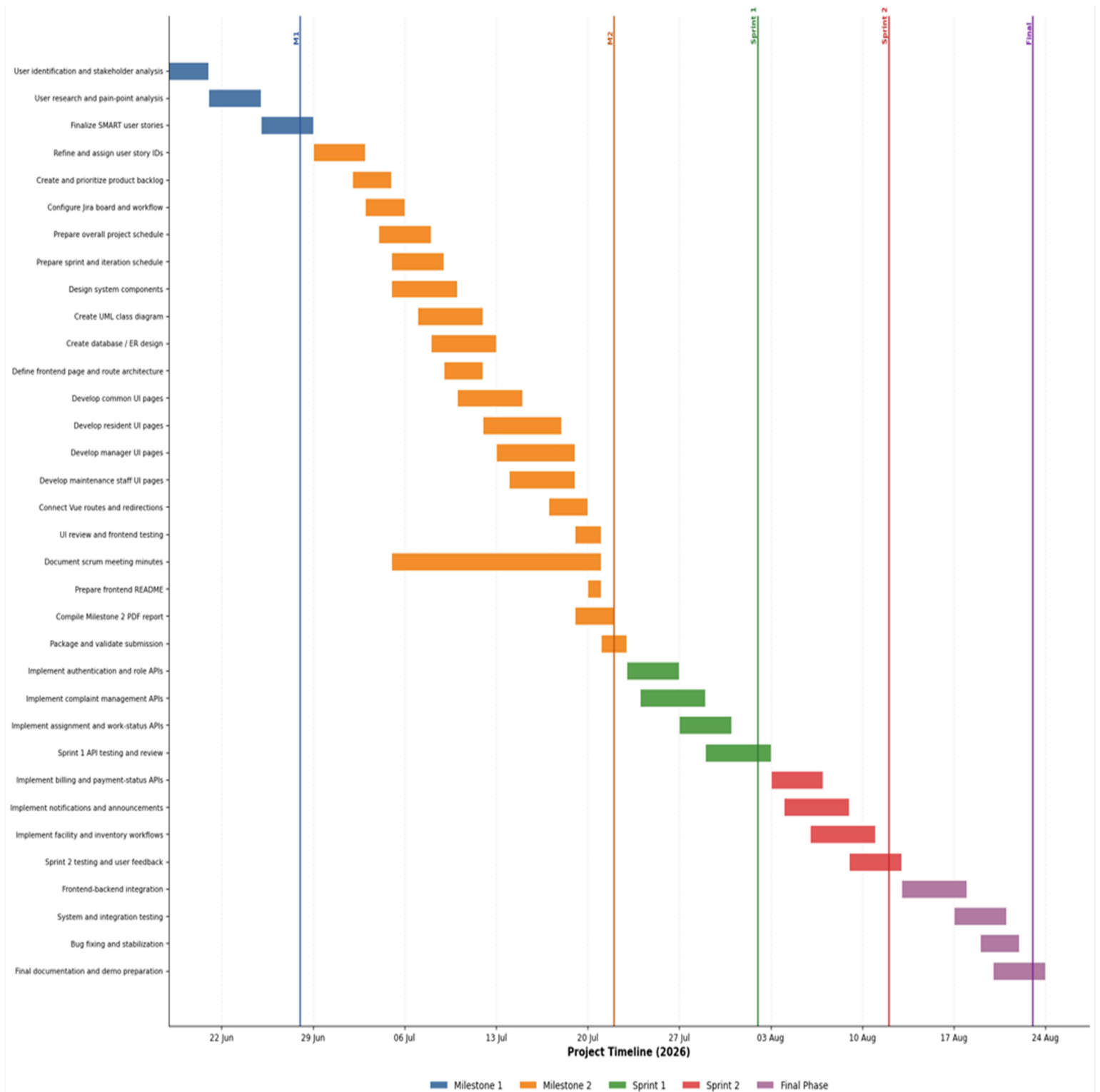


Figure 1: Gantt Chart for Smart Society Complaint Resolution System

The Gantt chart shown above illustrates the overall project schedule for the **Smart Society Complaint Resolution System**, covering all major milestones, sprint

activities, and final deliverables from **June 2026 to August 2026**. It provides a clear

Tasks	Starting Date	Completion Date
-------	---------------	-----------------

timeline of task execution, dependencies, and the planned duration of each phase.

The project begins with **Milestone 1**, focusing on requirements gathering, stakeholder analysis, user research, and the preparation of SMART user stories. **Milestone 2** emphasizes project scheduling, system design, Jira configuration, UML class diagram, database design, frontend development using **HTML, CSS, JavaScript (Vue via CDN)**, and project documentation.

Following the design phase, **Sprint 1** focuses on implementing the core backend APIs, API documentation, testing, and user feedback. **Sprint 2** extends the implementation with additional system features, API enhancements, and further testing based on user feedback. The **Final Phase** includes frontend-backend integration, system testing, bug fixing, documentation, and project demonstration.

The chart also highlights the use of the **Agile-SCRUM methodology**, where development activities are organized into milestones and iterative sprints, enabling effective project planning, task management, and timely completion of deliverables.

Color Legend:

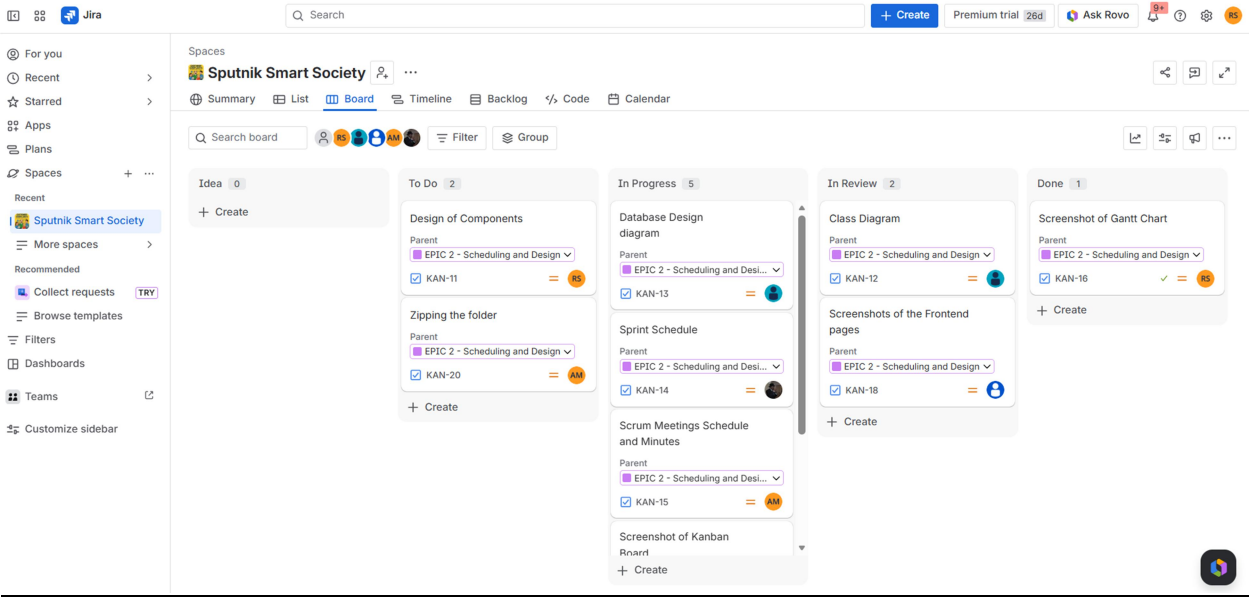
- **Blue:** Milestone 1 – Requirements Engineering
- **Orange:** Milestone 2 – Planning, Design, and Frontend Development
- **Green:** Sprint 1 – Core API Development
- **Red:** Sprint 2 – Feature Enhancement and Testing
- **Purple:** Final Integration, Testing, and Project Submission

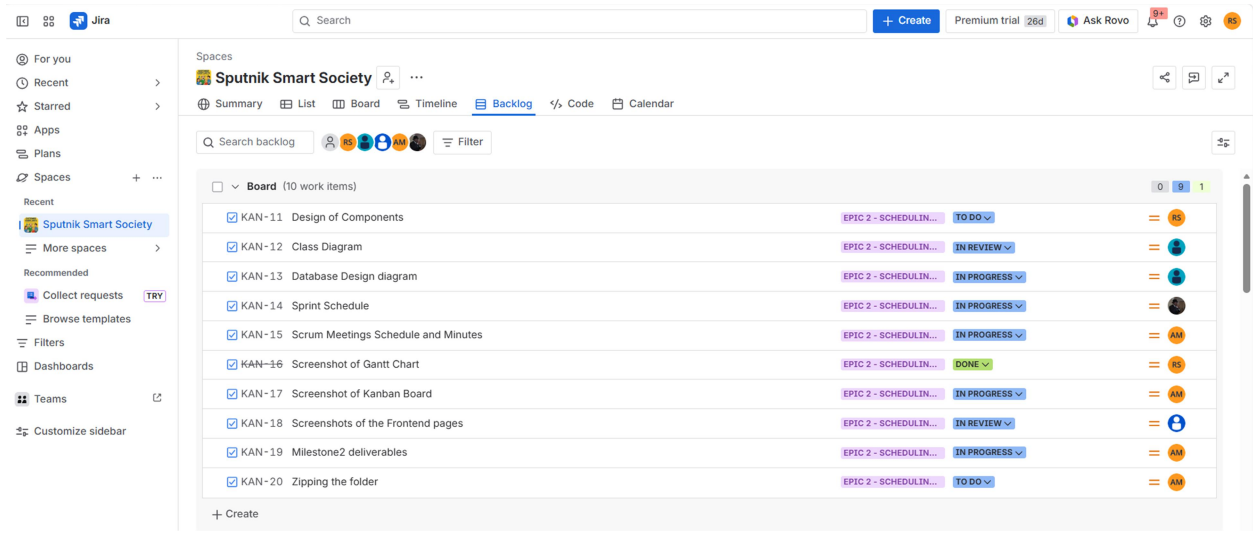
The tasks are further mentioned below in the excel table format for reference

Identify users and analyze stakeholders	12-Jun-26	15-Jun-26
Conduct user research and analyze pain points	16-Jun-26	20-Jun-26
Document user stories according to SMART guidelines	21-Jun-26	27-Jun-26
Submit Milestone 1	28-Jun-26	28-Jun-26
Prepare the overall project schedule	29-Jun-26	02-Jul-26
Prepare the Scrum meeting schedule	30-Jun-26	03-Jul-26
Prepare the sprint and iteration schedule	01-Jul-26	04-Jul-26
Prepare the Gantt chart	03-Jul-26	06-Jul-26
Configure Jira and assign the workflow	29-Jun-26	04-Jul-26
Define and document system components	04-Jul-26	08-Jul-26
Prepare the UML class diagram	07-Jul-26	11-Jul-26
Design the database and ER model	08-Jul-26	13-Jul-26
Define frontend page and route architecture	09-Jul-26	12-Jul-26
Develop common UI pages	11-Jul-26	15-Jul-26
Develop resident UI pages	13-Jul-26	18-Jul-26
Develop manager UI pages	14-Jul-26	19-Jul-26
Develop maintenance staff UI pages	15-Jul-26	19-Jul-26
Connect Vue routes and implement redirections	18-Jul-26	20-Jul-26
Document Scrum meeting minutes	30-Jun-26	21-Jul-26
Prepare the frontend README file	19-Jul-26	21-Jul-26
Compile the Milestone 2 PDF report	18-Jul-26	21-Jul-26
Submit Milestone 2 and the validity form with zipped Milestones 1 and 2	22-Jul-26	22-Jul-26
Design API endpoints for User Story 2	23-Jul-26	24-Jul-26
Prepare Swagger-compatible YAML documentation for the API	24-Jul-26	26-Jul-26
Implement and integrate the API	25-Jul-26	28-Jul-26
Design extensive test cases for each API endpoint	27-Jul-26	29-Jul-26
Test the API endpoints	29-Jul-26	31-Jul-26
Demonstrate features to end users, collect feedback, and plan the next sprint	31-Jul-26	01-Aug-26
Submit pytest code and Sprint 1 deliverables	02-Aug-26	02-Aug-26
Refine the User Story 2 API based on Sprint 1 feedback	03-Aug-26	04-Aug-26
Design API endpoints for User Story 3	03-Aug-26	05-Aug-26
Prepare Swagger-compatible YAML documentation for the API	04-Aug-26	06-Aug-26
Implement and integrate the API	05-Aug-26	08-Aug-26
Design extensive test cases for each API endpoint	07-Aug-26	09-Aug-26
Test the API endpoints	09-Aug-26	10-Aug-26
Demonstrate features to end users and collect feedback	10-Aug-26	11-Aug-26
Refine the User Story 3 API based on user feedback	11-Aug-26	11-Aug-26
Submit pytest code and Sprint 2 deliverables	12-Aug-26	12-Aug-26
Complete project implementation and working prototype	13-Aug-26	19-Aug-26
Prepare the final project report	16-Aug-26	21-Aug-26
Prepare the implementation README with user instructions	18-Aug-26	21-Aug-26
Review and update the issue tracker	13-Aug-26	22-Aug-26

Prepare the project presentation and demonstration	20-Aug-26	22-Aug-26
Submit the final project	23-Aug-26	23-Aug-26

SCREENSHOTS OF KANBAN BOARD(PARTIAL)





SCREENSHOTS OF FRONT END PAGES DEVELOPED(PARTIAL)

Access Portal

Sign in to the complaint resolution system.

SYSTEM ID / USERNAME

e.g., admin, resident

PASSWORD

[Recover](#)

Authenticate

Unregistered resident? [Request Access](#)



Complaint Resolution System

A dedicated platform for residents and staff to seamlessly track, manage, and resolve all society maintenance issues.

DEMO ACCESS CREDENTIALS

ADMINISTRATOR

U: admin
P: admin123

RESIDENT

U: resident
P: resident123

MAINTENANCE STAFF

U: maintenance
P: maintenance123

Reset Password

Enter your email to receive password recovery instructions.

EMAIL ADDRESS

your.email@example.com

Send Recovery Link

Remember your password? [Back to Login](#)

Request Access

Register as a new resident.

FIRST NAME

LAST NAME

UNIT NUMBER

EMAIL

Submit Application

[← Back to Login](#)



Password Recovery

We'll send a secure recovery link to your registered email address. Click the link to reset your password.



Join the Community

Submit your details and our admin team will verify your residency and grant access.



Smart Society
COMPLAINT RESOLUTION

 Alerts

John Doe
RESIDENT | UNIT B-101

J



Portal

Support

Resident Portal

Track your service requests and facility updates.

+ File New Request

TICKETS FILED
6

IN PROGRESS
3

ACTION REQUIRED
1 Verify Work

SUCCESSFULLY CLOSED
2

MY SERVICE LEDGER

TICKET ID	ISSUE DETAILS	PRIORITY	LIFECYCLE STATUS	DATE LOGGED	ACTION
TKT-1001	Plumbing Leakage / Blockage	High	IN PROGRESS	01/07/2026	View Details
TKT-1003	Electrical Power Outage	Low	CLOSED	10/05/2026	View Details
	Plumbing				



Smart Society
COMPLAINT RESOLUTION

 Alerts

John Doe
RESIDENT | UNIT B-101

J



TKT-1001

IN PROGRESS

HIGH PRIORITY

Leakage / Blockage

Print

Overview & Media

Discussions 3

FULL DESCRIPTION

Continuous water leaking from the main tap.

ATTACHED EVIDENCE



+ Add Media

Ticket Details

LOGGED BY

JD John Doe (B-101)
01/07/2026 09:30 AM

CATEGORY & LOCATION

Plumbing - Leakage / Blockage

Master Bathroom

ASSIGNED TO

MJ Michael Johnson
Maintenance Staff

Activity Timeline

Smart Society

COMPLAINT RESOLUTION

Alerts

John Doe

RESIDENT | UNIT B-101

J

←

TKT-1001

IN PROGRESS

HIGH PRIORITY

Leakage / Blockage

Print

Overview & Media

Discussions3

TICKET LOGGED - OCT 24

09:15 AM

RESIDENT

You

JD

Please send someone fast, the dripping is getting worse. I had to put a bucket underneath.

AU

Admin User

ADMIN

10:48 AM

Hi John, we have assigned Mike to this. He is currently checking flat 502 above you.

MS

Mike Smith

STAFF

11:10 AM

Admin, the issue is indeed from 502. The washing machine pipe burst. I've turned off their water supply temporarily. Going down to 402 now to assess ceiling damage.

Type your message...

Send

Ticket Details

LOGGED BY

JD

John Doe (B-101)

01/07/2026 09:30 AM

CATEGORY & LOCATION

Plumbing - Leakage / Blockage

Master Bathroom

ASSIGNED TO

MJ

Michael Johnson

Maintenance Staff

Activity Timeline

Work Commenced

01/07/2026 11:00 AM

Assigned Task

01/07/2026 10:15 AM

PROJECT SCHEDULE

Date	Phase/Focus	Main Agenda Points	Expected outcome
Before Jun 28	Milestone 1- Identify User Requirement	a. Identify users (primary, secondary, tertiary) b. Conduct interviews/observations to identify pain points (no direct feature suggestion) c. Record audio/video proof of research d. Write SMART user stories: As a [user], I want [action], so that [benefit] e. Compile deliverables (users, research, pain points, user stories in PDF)	Milestone 1 completed and ready for submission
Jun 29	Milestone 2 kick off	a. Kick off discussion for Milestone 2 b. Record meeting minutes (attendees, decisions, actions)	Milestone 2 started, minutes captured
Jul 4	Pre-Milestone 2 alignment & UI/tech prep	a. Confirm tech stack & open-source libraries b. Review digitized high-fidelity wireframes c. Validate UI designs against Milestone 1 user stories e. Record meeting minutes	Tech stack locked, wireframes validated & approved, minutes captured
Jul 9	Full project scheduling &	a. Finalize complete project timeline till Aug	

	alignment	23 deadline b. Create/export Gantt chart + high-level timeline (compressed) c. Define 2 short sprints clearly d. Set up Trello/Jira board + initial tasks e. Finalize tools/tech stack (Jira/Trello tracking, GitHub issues/PRs, Draw.io diagrams, Flask backend, Vue/React frontend, pytest, etc.) f. Record meeting minutes	Full A-to-Z schedule ready (ends August 23), Gantt created, board live, tools finalized, minutes captured
Jul 12	Milestone 2 design + UI	a. Review full schedule/Gantt (take/export screenshots for PDF) b. Finish class diagram + system components description c. Finish DB design (ER diagram/schema) d. Frontend: demo most linked UI pages (navigable) e. Export Jira screenshot for PDF f. Record meeting minutes	Design & UI ~80–90% done, Gantt + Jira screenshots ready, minutes captured
Jul 15	Milestone 2 polish + Sprint 1 prep	a. Final Milestone 2 check (diagrams, UI screenshots, zip/readme) b. Prepare PDF sections (schedule, Gantt/Jira screenshots, diagrams, UI pages, minutes so far) c. Sprint 1 priority	Milestone 2 95% ready, PDF sections drafted, minutes captured

		APIs/user stories & assignments d. Risk discussion (tight timeline → plan daily async standups via Slack if needed) e. Record meeting minutes	
Jul 21	Milestone 2 closure + Milestone 3 kickoff	a. Final verification of all Milestone 2 deliverables b. Final "sanity check" of code/documentation c. Kickoff Milestone 3 (Sprint 1: API dev) d. Record meeting minutes	Milestone 2 submitted, Sprint 1 started, minutes captured
Jul 22	Sprint 1 – API progress & mid-check	a. Standup: progress last week b. Demo 4–6 API endpoints (Postman/YAML) c. Review PRs/issues d. Update board & Gantt e. Log blockers/bugs as GitHub issues (with screenshots for final report) f. Record meeting minutes	~50% Sprint 1 APIs done, issues tracked, minutes captured
Jul 30	Sprint 1 – Wrap-up & testing focus	a. Final Sprint 1 demos (all APIs) b. YAML completeness & user story mapping check c. Run pytest suite (show results/screenshots) d. Log all bugs as GitHub issues (with status updates) e. Quick retro + Sprint 2	Sprint 1 complete: APIs + YAML + basic tests ready, issues tracked, minutes captured

		handoff f. Record meeting minutes	
Aug 3	Sprint 2 kickoff – Integration & testing start	a. Kickoff Sprint 2: connect frontend-backend b. Assign integration + test tasks c. Plan test cases matrix (input/expected/actual) d. Update board with Sprint 2 backlog e. Quick refresh (CORS, auth if needed) f. Record meeting minutes	Sprint 2 started, first integrations attempted, minutes captured
Aug 12	Sprint 2 – Final testing, polish & submission prep	a. Demo full integrated flows b. Show test cases (4–5 APIs) + pytest results c. Highlight failures + fixes (document for report) d. Gather quick user feedback (document summary + plan for final) e. Final README check (include hosting info if deployed, e.g., Render) f. Compile final PDF report (M2 + Sprint 1 + Sprint 2 + tools + GitHub issue screenshots) g. Record draft video demo (5–10 min overview + user stories) h. Start slides (overview, user stories, design/teamwork) i. Dry-run demo & submit prep	App stable, tests ready, report/video/slides draft done, minutes captured → Submit

		j. Record meeting minutes	
Aug 23	Milestone 5 (Final submission)	a. Verify and do a final sanity check of the whole project b. Record presentation video, Finalize video (edit/upload) c. Finish PDF report (proofread, add run instructions) - Final code zip + README (install/run steps for Ubuntu/Mac + Windows) - d. Submit all (zip, PDF, video, slides)	The whole Software engineering project including all the code zip, documentation, milestones submitted in the submission drive

SCRUM Meetings Schedule and Minutes

SCRUM Meetings: Conducted on scheduled dates via Google Meet (evening sessions)

Minutes from June 20, 2026 SCRUM Meeting:

- Conducted team introductions and discussed individual roles and expertise
- Analyzed multiple project domains and finalized Apartment Association/Issue tracking
- Brainstormed issue tracking system workflow
- Identified key pain points such as delayed issue resolution, lack of transparency, poor communication, no accountability

Minutes from June 24, 2026 SCRUM Meeting:

- Reviewed team progress and clarified individual responsibilities
- Discussed compliance requirements including audio/video consent

- Detailed discussion about the design architecture
- Introduced SMART guidelines for writing user stories
- Identified and finalized primary, secondary, and tertiary users

Minutes from June 26, 2026 SCRUM Meeting:

- Analyzed interview data and documented key pain points
- Drafted 17 SMART user stories covering the complete workflow
- Discussed sanity check of documentation and user stories
- Initiated Milestone 2 with focus on frontend wireframing and UI structure

Minutes from July 1, 2026 SCRUM Meeting:

- Confirmed technology stack (Flask, Vue, Tailwind CSS, etc.)
- Reviewed and validated UI wireframes
- Proposed UI designs for development phase
- Discussed about the duration of the workflow and sprint schedule

Minutes from July 3, 2026 SCRUM Meeting:

- Frontend template was presented and reviewed by the team.
- Jira project structure, task tracking, and workflow were discussed.
- Importance of maintaining the GitHub repository was emphasized
- Team agreed to develop the frontend and backend in parallel

Minutes from July 8, 2026 SCRUM Meeting:

- Team discussed the APIs required for communication between the frontend and backend

- The distinction between the class diagram and the database design (ER) diagram was clarified
- Current progress of the frontend development was reviewed
- The frontend folder structure was discussed, focusing on organizing components, pages, etc.

Minutes from July 9, 2026 SCRUM Meeting:

- Discussed about the APIs to be implemented in the coming sprints
- Drafts of Gantt chart, minutes, class diagram, Database design diagram were documented
- Few APIs were proposed and discussed
- Doubts regarding the future milestones were discussed
- Agreed to come up with the collection of milestone 2 documents

PROJECT SCHEDULING TOOLS

To manage the project effectively, we used visual task management and scheduling tools to organize work across different phases and ensure timely completion of milestones.

Tools Used:

- Jira (for task management and tracking)
- Gantt Chart (Excel) (for project timeline visualization)

Description:

We primarily used a Jira to manage and track the progress of tasks throughout the project. Tasks were categorized into columns such as To Do, In Progress, and Completed, allowing the team to clearly visualize the workflow and monitor progress at each

stage of development. This improved transparency and ensured that all team members were aware of their responsibilities.

Additionally, a Gantt chart was created to represent the complete project schedule from June to August. This provided a high-level view of milestones, deadlines, and dependencies, ensuring that all phases such as design, API development, integration, and testing were completed on time.

Outcome:

These tools helped in:

- Efficient tracking of tasks and progress
- Clear visualization of workflow
- Proper documentation of issues
- Timely completion of all milestones

Gantt Chart

The Gantt chart represents the complete timeline of the project from Milestone 1 (User Requirement Analysis) to Final Submission (August 23, 2026). It provides a visual representation of all major phases, including planning, design, development, integration, testing, and final delivery.

The chart illustrates how the project is divided into multiple milestones and sprints, ensuring a structured and systematic approach to development.

Description

The project begins with Milestone 1, which focuses on identifying user requirements through interviews, observations, and formulation of SMART user stories. This phase lays the foundation for all subsequent development activities.

Following this, Milestone 2 covers planning and design activities, including project

scheduling, Gantt chart creation, system design (class diagrams and database schema), and UI development. This phase ensures that the architecture and workflow of the system are clearly defined before implementation begins.

The development phase is carried out in Sprint 1 (Milestone 3), where core APIs are designed, implemented, and tested. Tasks are divided into initial API development and completion with testing, ensuring incremental progress.

Next, Sprint 2 (Milestone 4) focuses on integrating the frontend and backend components. This phase includes implementing core user flows, conducting detailed testing, fixing bugs, and collecting feedback.

Finally, Milestone 5 involves final testing, report preparation, video demonstration, and submission of all deliverables.

Key features of Gantt Chart

- Clearly shows the timeline of all milestones and sprints
- Represents parallel and overlapping tasks (e.g., UI development and system design)
- Highlights dependencies between phases (design → development → testing → submission)
- Ensures efficient time management and task tracking
- Helps in monitoring progress and meeting deadlines

Outcome

The Gantt chart played a crucial role in:

- Planning the project effectively
- Managing task dependencies and overlaps
- Tracking progress across milestones and sprints
- Ensuring timely completion of the project within the deadline

PROJECT TRACKING MECHANISM

(JIRA)

To effectively manage and track project progress, a Jira was used as the primary project tracking tool. It provided a visual representation of tasks across different stages of development, ensuring transparency and efficient workflow management.

Description:

Jira was structured into multiple columns representing task status:

- Backlog – Tasks that are planned but not yet started
- To Do – Tasks ready to be picked up
- In Progress – Tasks currently being worked on
- In Review – Tasks under review or testing
- Done – Completed tasks

Each task was represented as a card containing details such as task title, priority, and assignee. Tasks were continuously moved across columns as progress was made.

Jira also included:

- Task prioritization
- Assignment of responsibilities to team members
- Real-time progress tracking

Outcome:

Jira helped in:

- Efficient task tracking and management

- Clear visualization of project progress
- Better team coordination and accountability
- Timely completion of milestones and deliverables

Jira showing task distribution and progress tracking across different stages

The screenshot displays a Jira Kanban board for the 'Sputnik Smart Society' project. The board is organized into five columns representing different stages of the workflow: Idea (5 tasks), To Do (3 tasks), In Progress (2 tasks), In Review (0 tasks), and Done (0 tasks). Each task card includes a title, parent epic, assignee, and status. For example, in the 'To Do' column, tasks include 'Design of Components', 'Sprint Schedule', and 'Scrum Meetings Schedule and Minutes'. The 'In Progress' column shows 'Class Diagram' and 'Database Design diagram'. The 'In Review' and 'Done' columns are currently empty.

Below the Kanban board, a table provides a detailed view of the tasks, including their IDs, titles, assignees, reporters, priorities, statuses, resolutions, creation dates, and update dates.

	Work	Assignee	Reporter	Priority	Status	Resolution	Created	Update
☐	> KAN-4 EPIC 1 - Identify User Requirements	P Baskaran Nadar	ANNAPOORANI...	Medium	DONE	Done	Jul 07, 2026, 1:06 PM	Jul 09, 2026,
☐	> KAN-5 EPIC 2 - Scheduling and Design	P Baskaran Nadar	ANNAPOORANI...	Medium	IN PROGRESS	Unresolved	Jul 07, 2026, 1:08 PM	Jul 09, 2026,
☐	> KAN-21 EPIC 3 - Define APIs	24f1002269 - Rahul ...	P Baskaran Nadar	Medium	IN REVIEW	Unresolved	Jul 07, 2026, 6:47 PM	Jul 09, 2026,
☐	> KAN-32 EPIC 4 - API Development and Testi	P Baskaran Nadar	P Baskaran Nadar	Medium	TO DO	Unresolved	Jul 09, 2026, 12:13 PM	Jul 09, 2026,
☐	> KAN-33 EPIC 5 - Sprint 2 API Refinement and Expan...	P Baskaran Nadar	P Baskaran Nadar	Medium	IDEA	Unresolved	Jul 09, 2026, 12:21 PM	Jul 09, 2026,
☐	KAN-64 EPIC 6 - Develop 'Stubs' for backend API calls	P Baskaran Nadar	P Baskaran Nadar	Medium	IN REVIEW	Unresolved	Jul 10, 2026, 2:20 PM	Jul 10, 2026,
☐	> KAN-63 Milestone 2	P Baskaran Nadar	P Baskaran Nadar	Medium	IDEA	Unresolved	Jul 09, 2026, 1:18 PM	Jul 09, 2026,

Spaces

Sputnik Smart Society

Summary

List

Board

Timeline

Backlog

Code

Calendar

Search backlog

Filter

10 of 10 work items visible

Backlog

(35 work items)

32

3

0

☒	KAN - 24	EPIC 3 - Define Backend requirements	EPIC 3 - DEFINE APIS	IN PROGRESS	Jul 11	
☒	KAN - 23	EPIC 3 - Define Frontend requirements	EPIC 3 - DEFINE APIS	IN PROGRESS	Jul 11	
☒	KAN - 25	Design API endpoints for User Story 2	EPIC 4 - API DEVELO...	IDEA		
☒	KAN - 26	Prepare Swagger-compatible YAML documentation for the API	EPIC 4 - API DEVELO...	IN REVIEW	Jul 9	
☒	KAN - 27	Implement and integrate the API	EPIC 4 - API DEVELO...	IDEA		
☒	KAN - 28	Design extensive test cases for each API endpoint	EPIC 4 - API DEVELO...	IDEA		
☒	KAN - 29	Test the API endpoints	EPIC 4 - API DEVELO...	IDEA		
☒	KAN - 30	Demonstrate features to end users, collect feedback, and plan the next sprint	EPIC 4 - API DEVELO...	IDEA		
☒	KAN - 31	Submit pytest code and Sprint 1 deliverables	EPIC 4 - API DEVELO...	IDEA		
☒	KAN - 34	Refine the User Story 2 API based on Sprint 1 feedback	EPIC 5 - SPRINT 2 AP...	IDEA		
☒	KAN - 37	Design API endpoints for User Story 3	EPIC 5 - SPRINT 2 AP...	IDEA		
☒	KAN - 39	Implement and integrate the API	EPIC 5 - SPRINT 2 AP...	IDEA		

Spaces

Sputnik Smart Society

Summary

List

Board

Timeline

Backlog

Code

Calendar

Search timeline

Filter

	Work	Status	Assignee	June	July	August
☐	> KAN-4 EPIC 1 - Identify User Requirements		P Baskaran Nadar			
☐	> KAN-5 EPIC 2 - Scheduling and Design		P Baskaran Nadar			
☐	> KAN-21 EPIC 3 - Define APIs		24f1002269 - Ra...			
☐	> KAN-32 EPIC 4 - API Development and Testi		P Baskaran Nadar			
☐	> KAN-33 EPIC 5 - Sprint 2 API Refinement and E...		P Baskaran Nadar			
☐	> KAN-64 EPIC 6 - Develop 'Stubs' for backend A...	IN REVIEW	P Baskaran Nadar			
☐	> KAN-63 Milestone 2		P Baskaran Nadar			
+ Create				7 of 7		

Today

Weeks

Months

Quarters

FRONTEND INTERFACE SCREENS

Access Portal

Sign in to the complaint resolution system.

SYSTEM ID / USERNAME

e.g., admin, resident

PASSWORD

[Recover](#)

Authenticate

Unregistered resident? [Request Access](#)

Reset Password

Enter your email to receive password recovery instructions.

EMAIL ADDRESS

your.email@example.com

Send Recovery Link

Remember your password? [Back to Login](#)



Complaint Resolution System

A dedicated platform for residents and staff to seamlessly track, manage, and resolve all society maintenance issues.

DEMO ACCESS CREDENTIALS

ADMINISTRATOR

U: admin
P: admin123

RESIDENT

U: resident
P: resident123

MAINTENANCE STAFF

U: maintenance
P: maintenance123



Password Recovery

We'll send a secure recovery link to your registered email address. Click the link to reset your password.

Request Access

Register as a new resident.

FIRST NAME

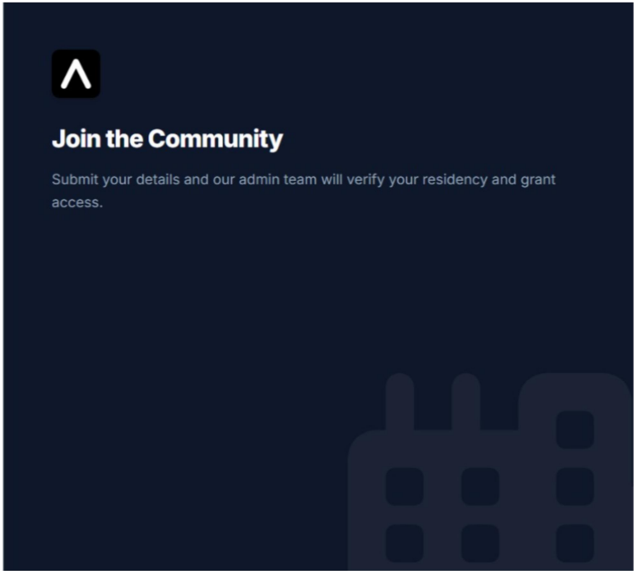
LAST NAME

UNIT NUMBER

EMAIL

Submit Application

← Back to Login



Smart Society
COMPLAINT RESOLUTION

Alerts

John Doe
RESIDENT | UNIT B-101

J

Portal

Support

Resident Portal

+ File New Request

TICKETS FILED
6

IN PROGRESS
3

ACTION REQUIRED
1 Verify Work

SUCCESSFULLY CLOSED
2

MY SERVICE LEDGER

TICKET ID	ISSUE DETAILS	PRIORITY	LIFECYCLE STATUS	DATE LOGGED	ACTION
TKT-1001	Plumbing Leakage / Blockage	High	IN PROGRESS	01/07/2026	View Details
TKT-1003	Electrical Power Outage	Low	CLOSED	10/05/2026	View Details
	Plumbing				

← Log Service Request

TRADE CATEGORY *

Select category...

SPECIFIC ISSUE *

Select specific issue...

IMPACT / PRIORITY *

Low

Medium

High

EXACT LOCATION *

e.g. Master Bath, Kitchen

DETAILED DESCRIPTION *

Provide details to assist the maintenance team...

Cancel

Submit Request



TKT-1001

IN PROGRESS

HIGH PRIORITY

Leakage / Blockage

 Print

 Overview & Media

 Discussions 3

FULL DESCRIPTION

Continuous water leaking from the main tap.

ATTACHED EVIDENCE

+ Add Media



Ticket Details

LOGGED BY

 **John Doe (B-101)**
01/07/2026 09:30 AM

CATEGORY & LOCATION

 Plumbing - Leakage / Blockage

 Master Bathroom

ASSIGNED TO

 **Michael Johnson**
Maintenance Staff

Activity Timeline

Smart Society

COMPLAINT RESOLUTION

Alerts

John Doe

RESIDENT | UNIT B-101

J

←

TKT-1001

IN PROGRESS

HIGH PRIORITY

Leakage / Blockage

Print

Overview & Media

Discussions 3

TICKET LOGGED - OCT 24

09:15 AM

RESIDENT

You

JD

Please send someone fast, the dripping is getting worse. I had to put a bucket underneath.

AU

Admin User

ADMIN

10:48 AM

Hi John, we have assigned Mike to this. He is currently checking flat 502 above you.

MS

Mike Smith

STAFF

11:10 AM

Admin, the issue is indeed from 502. The washing machine pipe burst. I've turned off their water supply temporarily. Going down to 402 now to assess ceiling damage.

Type your message...

Send ↗

Ticket Details

LOGGED BY

JD

John Doe (B-101)

01/07/2026 09:30 AM

CATEGORY & LOCATION

🔧 Plumbing - Leakage / Blockage

📍 Master Bathroom

ASSIGNED TO

MJ

Michael Johnson

Maintenance Staff

Activity Timeline

● Work Commenced

01/07/2026 11:00 AM

● Assigned Task

01/07/2026 10:15 AM